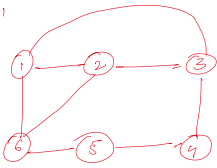


Hamiltonian Cycle

Sunday, 15 February 2026 2:44 PM

* If a graph is given then we have to start from some starting vertex and visit all the vertices exactly once & return back to the starting vertex.

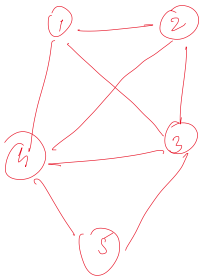
Ex: 1



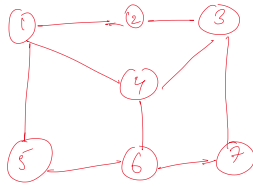
1, 2, 3, 4, 5, 6, 1
1, 2, 6, 5, 4, 3, 1
1, 6, 2, 5, 4, 3, 1
1, 3, 4, 5, 6, 1, 2

These are some cycles.

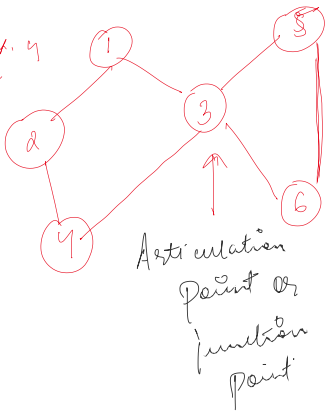
Ex: 2



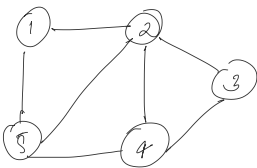
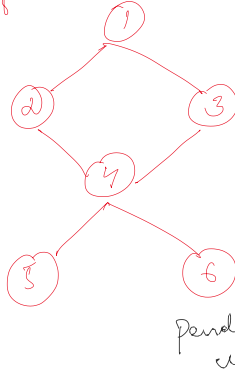
Ex: 3



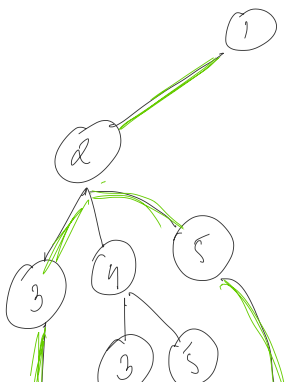
Ex: 4



Ex: 5



$$G = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$n \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix}$$


* Bounding function

- (1) no duplicates
- (2) there should be an edge to previous vertex
- (3) last vertex should be 'first' edge to first

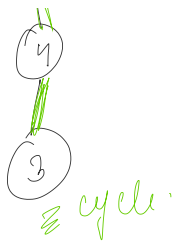
Algorithm Hamiltonian(k)

```

d
do
d
NextVertex(k);
if (x[k] == 0)
return;
if (k == n)
Print(x[1:n]);
Hamiltonian(k+1);
while(true);

```

Algorithm NextVertex(k)



```

    if (head == null) return null;
    do
    {
        x[k] = (x[k] + 1) % (n + 1);
        if (x[k] == 0) return;
        if (G[x[k-1], x[k]] != 0)
        {
            for (j = 1 to k-1 do if (x[j] == x[k])
                break;
            if (j == k)
            {
                if (k < n or (k == n) && G[x[n], x[1]] != 0)
                {
                    return; // last position
                }
                if edge back to vertex 1.
            }
        }
    } while (true);
}

```

② check if edge is there to previous
 ③ check for duplicate